ECE30030/ITP30010 Database Systems

ER-Model & Normalization

Reading: Chapter 3

Charmgil Hong

charmgil@handong.edu

Spring, 2026

Handong Global University

Announcements

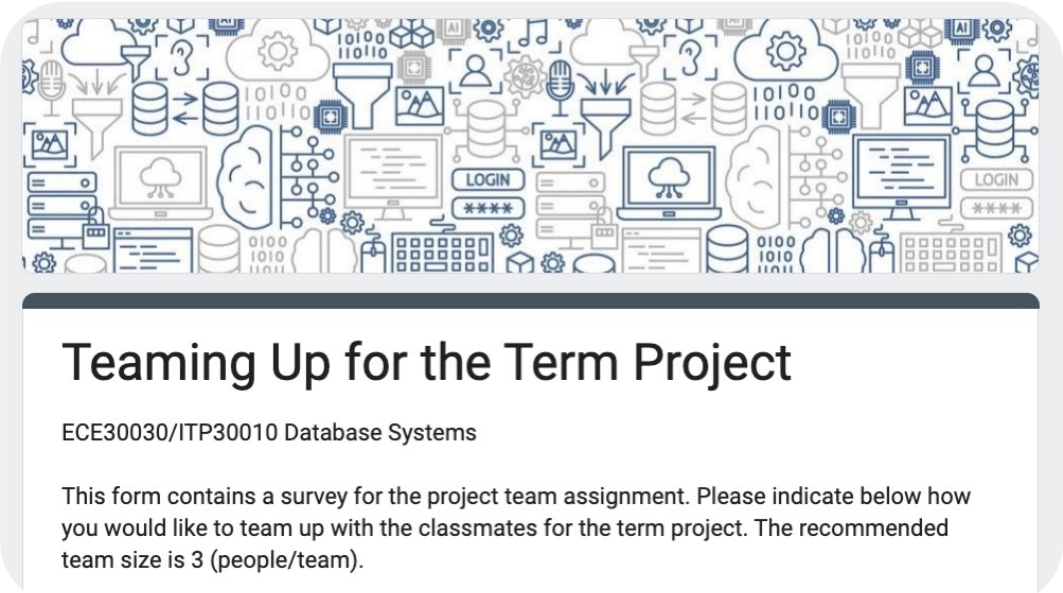
- Study material #4 is posted

- Quiz #3 is on April 23, 2026 (next Thursday)
  - Topics
    - 3 problems on SQL
    - 1 problem on ER-Model

- Midterm is on April 27, 2026 (Monday, Week #9)
  - Regular class hours
  - Closed-book
  - Topics
    - Relational data models
    - Relational algebra
    - Structured query language (DML, DDL)
    - Database design (ER-model, Normalization)

Announcements

- Make teams for the term project
  - https://forms.gle/mYJW7xU86Sv5ZD7H7 Reponse due: Monday, April 20
  - Problem & data will be released soon



Announcements

- TA office hours are available on a regular basis
  - Monday 14:00-15:00 -- Eunchan Jung (NTH217; 22100685@handong.ac.kr)
  - Tuesday 19:00-20:00 -- Hyunmo Kang (NTH217; 22100026@handong.ac.kr)
  - Thursday 14:00-15:00 -- Yunseong Choi (NTH221; yunseong.choi@handong.ac.kr)
  - Friday 14:00-15:00 -- Yujin Kim (NTH221; yujin.kim@handong.ac.kr) – won't be here this week

  - If you find any part of the material challenging or would like additional help, please feel encouraged to visit

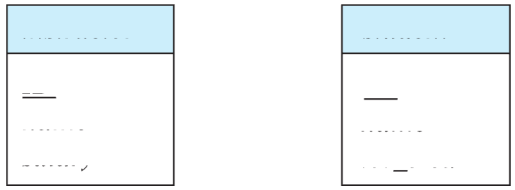
E-R Model for Database Modeling

- The E-R data model was developed to facilitate database design by allowing specification of a database schema
  - Database schema represents the overall logical structure of a database

- The E-R data model employs three basic concepts:
  - Entity sets
  - Relationship sets
  - Attributes

- The E-R model has an associated diagrammatic representation
  - E-R diagram can express the overall logical structure of a database graphically

Entity Sets



- An entity is an object that exists and is distinguishable from other objects
  - *E.g.*, specific person, company, event, plant
- An entity set is a set of entities of the same type that share the same properties
  - *E.g.*, set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; *i.e.*, descriptive properties possessed by all members of an entity set
  - *E.g.*, *instructor* = (*ID, name, salary*)
    course= (*course_id, title, credits*)
- A subset of the attributes form a primary key of the entity set; *i.e.*, uniquely identifying each member of the set

Relationship Sets

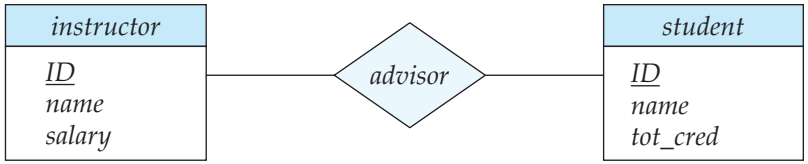
- A relationship is an association among several entities
  - *E.g.*,

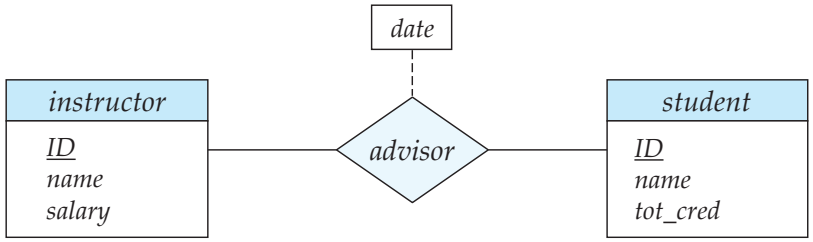
44553 (Peltier)	*advisor*	22222 (Einstein)
student entity	relationship set	*instructor* entity

- A relationship set is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \ldots e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

where $(e_1, e_2, \ldots, e_n)$ is a relationship
  - *E.g.*, (44553,22222) $\in$ *advisor*

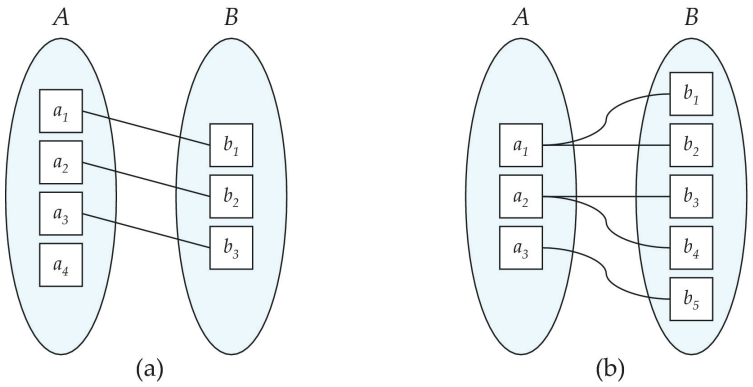


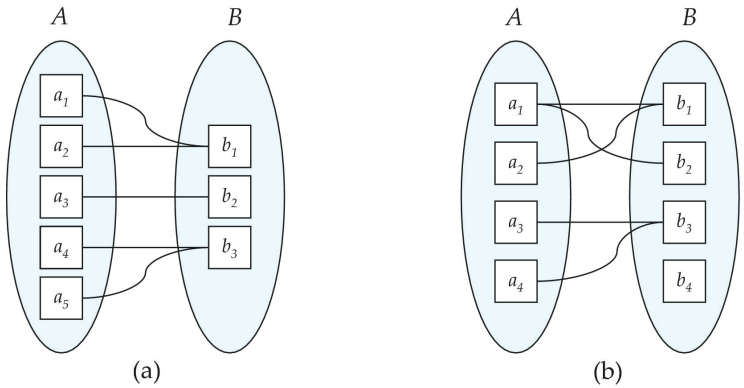


Mapping Cardinalities

- Express the number of entities to which another entity can be associated via a relationship set
  - Most useful in describing binary relationship sets

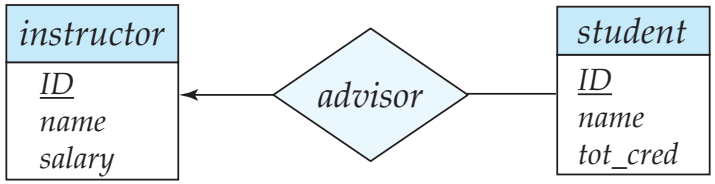
- For a binary relationship set the mapping cardinality must be one of the following types:
  - *One to one*
  - *One to many*
  - *Many to one*
  - *Many to many*

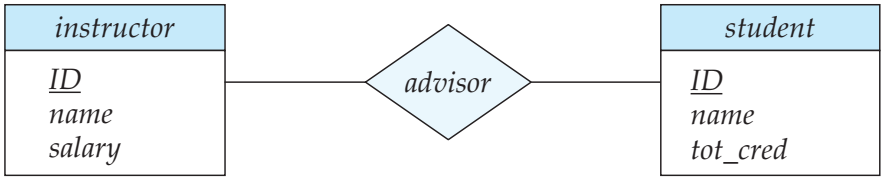




Representing Cardinalities in E-R Diagrams

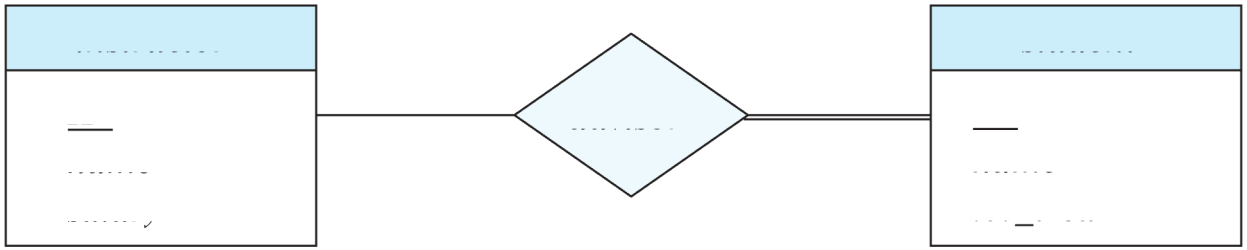
- Express cardinality constraints by drawing either a directed line (→), signifying "one," or an undirected line (—), signifying "many," between the relationship set and the entity set
  - One-to-one relationship between an *instructor* and a *student*
  - One-to-many relationship between an *instructor* and a *student*
  - Many-to-one relationship between an *instructor* and a *student*
  - Many-to-many relationship between an *instructor* and a *student*





Total and Partial Participation

- Total participation (*indicated by double line*): every entity in an entity set participates in at least one relationship in the relationship set



participation of *student* in *advisor r*elation is total

  - *E.g.*, Every *student* must have an associated *instructor*

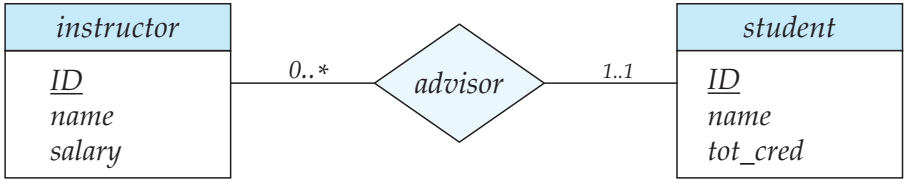
- Partial participation: some entities may not participate in any relationship in the relationship set

  - *E.g.*, Participation of *instructor* in *advisor* is partial

Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form *l..h*, where *l* is the minimum and *h* the maximum cardinality
  - A minimum value of 1 indicates total participation
  - A maximum value of 1 indicates that the entity participates in at most one relationship
  - A maximum value of * indicates no limit

- Examples
  - Instructor can advise 0 or more students
  - A student must have 1 advisor; cannot have multiple advisors



Agenda

- SQL data definition language (DDL)
- Designing a database
- E-R diagrams
  - Mapping cardinalities
  - **Primary keys in E-R models**
  - Weak entity sets
  - Reduction to relation schemas

Primary Key for Entity Sets

- By definition, individual entities are distinct
- From database perspective, the *differences among entities must be expressed in terms of their attributes*
  - The attribute values of an entity must be such that they can uniquely identify the entity
  - No two entities in an entity set are allowed to have exactly the same value for all attributes
- A key for an entity is a set of attributes that suffice to distinguish entities from each other

Primary Key for Relationship Sets

- To distinguish among the various relationships of a relationship set, use the individual primary keys of the entities in the relationship set
  - Let *R* be a relationship set involving entity sets $E_1, E_2, \ldots, E_n$
  - The primary key for *R* is consists of the union of the primary keys of entity sets $E_1, E_2, \ldots, E_n$
  - If the relationship set *R* has attributes $a_1, a_2, \ldots, a_m$ associated with it, then the primary key of *R* also includes the attributes $a_1, a_2, \ldots, a_m$

- Example: relationship set "*advisor*"
  - The primary key consists of *inrsructor.ID* and *student.ID*

Choice of Primary Key for Binary Relationship

- The choice of the primary key for a relationship set depends on the mapping cardinality of the relationship set
  - Many-to-Many relationships: The preceding union of the primary keys is a minimal super key and is chosen as the primary key
  - One-to-Many relationships: The primary key of the "Many" side is a minimal super key and is used as the primary key
    - Many-to-one relationships: The primary key of the "Many" side is a minimal super key and is used as the primary key
  - One-to-one relationships: The primary key of either one of the participating entity sets forms a minimal super key, and either one can be chosen as the primary key

Agenda

- SQL data definition language (DDL)
- Designing a database
- E-R diagrams
  - Mapping cardinalities
  - Primary keys in E-R models
  - **Weak entity sets**
  - Reduction to relation schemas

Weak Entity Sets

- A weak entity set is one whose existence is dependent on another entity, called its identifying entity
- Instead of associating a primary key with a weak entity, use the identifying entity, along with extra attributes called discriminator to uniquely identify a weak entity
  - A weak entity set does not have a primary key
  - We still need a *means of distinguishing* among an entity set
    - Discriminator of a weak entity: a set of attributes allowing such distinction
    - Primary key of a weak entity set
      = primary key of a strong entity set (which its existence depends)
      + its discriminator

Weak Entity Sets

- A weak entity set is one whose existence is dependent on another entity, called its identifying entity
- Instead of associating a primary key with a weak entity, use the identifying entity, along with extra attributes called discriminator to uniquely identify a weak entity
  - *E.g.*, Consider a *section* entity, which is uniquely identified by a *course_id*, *semester, year*, and *sec_id* → Section entities are related to course entities
    - Treat the relationship *sec_course* as a special relationship that provides extra information
    - In this case, the *course_id*, required to identify *section* entities uniquely

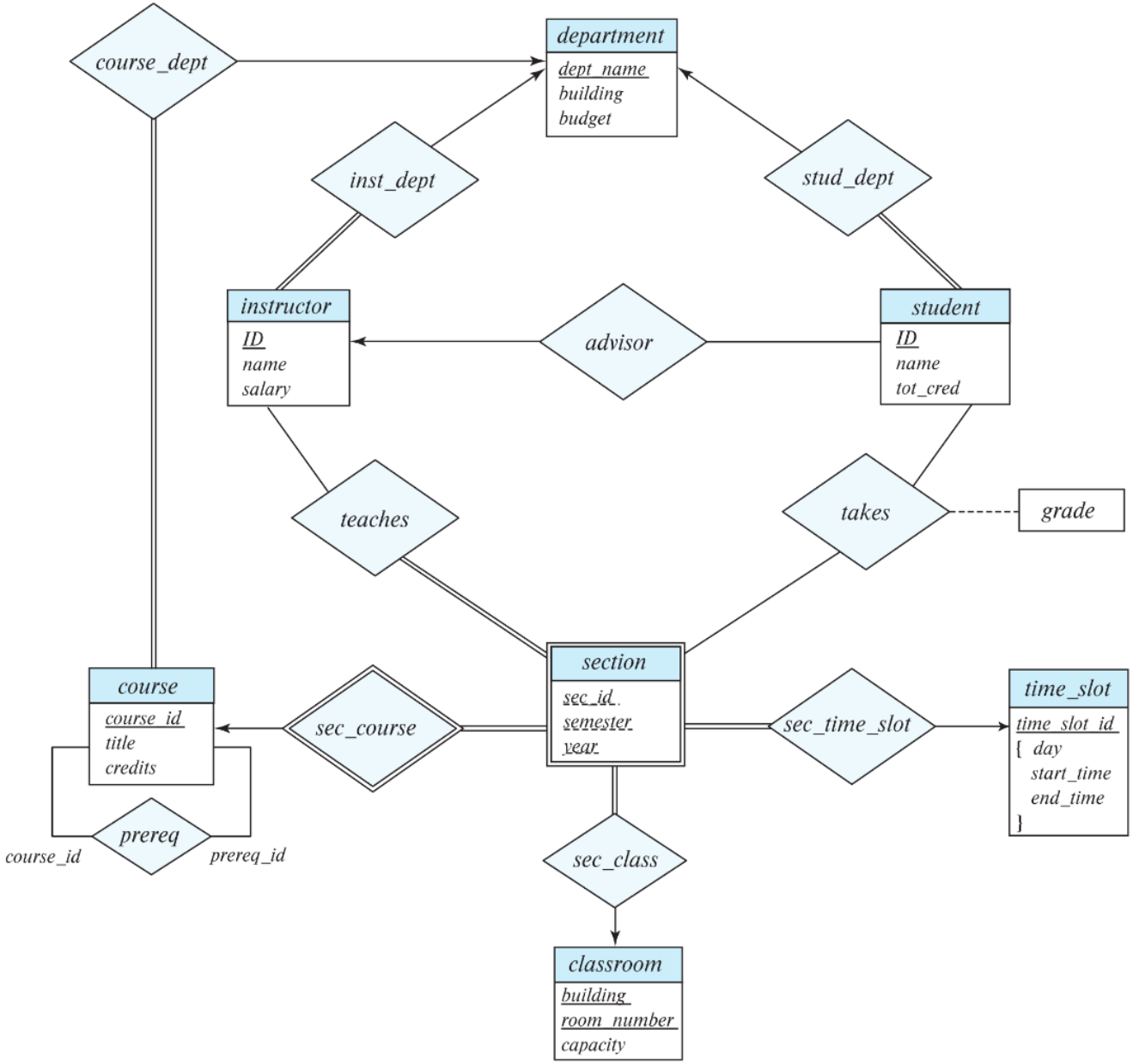
Weak Entity Sets

- Identifying entity
  - Every weak entity must be associated with an identifying entity;
    - That is, the weak entity set is said to be existence dependent on the identifying entity set

- The identifying entity set is said to own the weak entity set that it identifies
  - Identifying entity set: an entity set that has a primary key
  - Identifying entity set = strong entity set

- Identifying relationship
  - Identifying relationship: The relationship associating the weak entity set with the identifying entity set

E-R Diagram for a *University* Database



Example: a Record Shop

- Entity sets
  - *customer*
  - *product* (CD, vinyl)
  - *purchase*
- Relationship sets
  - *contains* (between *product* and *purchase*)
  - *buyer* (between *customer* and *purchase*)

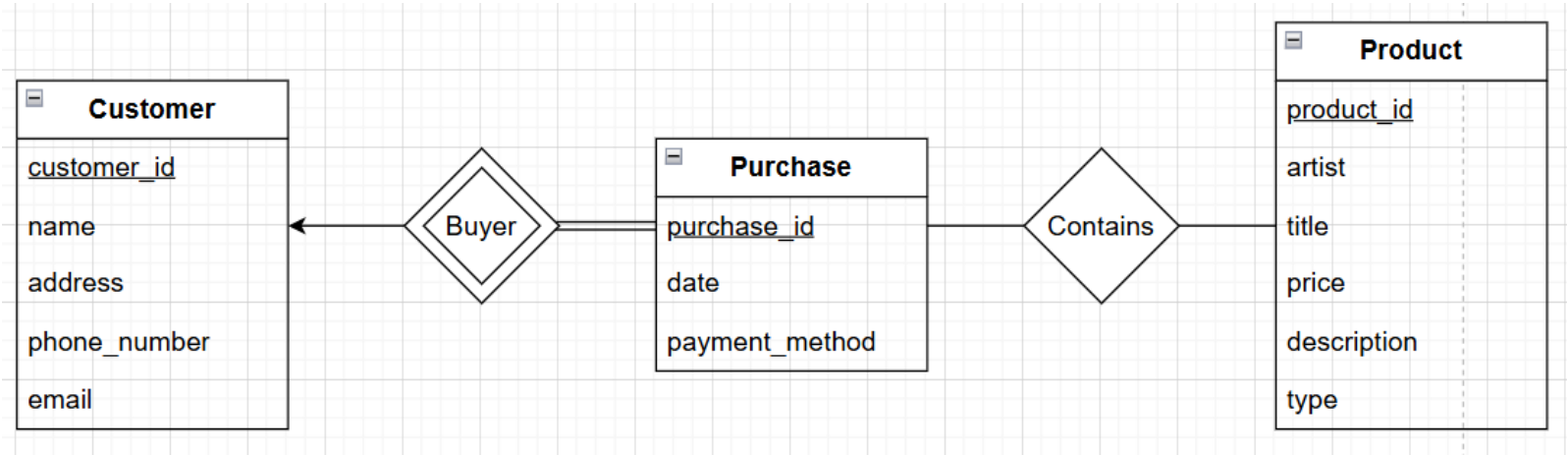
- An entity is a specific object; *E.g.*, a newest CD of BTS
- A relationship is specific pair of related objects

Example: a Record Shop

- Attributes
  - *customer*
    - *name, address, phone number, email, etc.*
  - *product*
    - *artist, title, price, description*
  - purchase
    - *date, payment_method*
  - Artificial primary keys for all entity sets (*customer_id*, *product_id*, *purchase_id*)

- Types of Relationships
  - 1-to-1
  - 1-to-many
    - *buyer* relation (between *customer* and *purchase*)
  - Many-to-many
    - *contains* relation (between *purchase* and *product*)

E-R Diagram


Customer
customer_id
name
address
phone_number
email
Buyer
Purchase
purchase_id
date
payment_method
Contains
Product
product_id
artist
title
price
description
type

ECE30030/ITP30010 Database Systems

Normalization

Reading: Chapter 7

Charmgil Hong

charmgil@handong.edu

Spring, 2026

Handong Global University

Agenda

- Motivating example
- Normal forms

When Data is Jumbled…

- Suppose that we combine *instructor* and *department*
  - (Below represents the join on *instructor* and *department)*

ID	name	salary	dept_name	building	budget
76766	Crick	72000.00	Biology	Watson	90000.00
10101	Srinivasan	65000.00	Comp. Sci.	Taylor	100000.00
45565	Katz	75000.00	Comp. Sci.	Taylor	100000.00
83821	Brandt	92000.00	Comp. Sci.	Taylor	100000.00
98345	Kim	80000.00	Elec. Eng.	Taylor	85000.00
12121	Wu	90000.00	Finance	Painter	120000.00
76543	Singh	80000.00	Finance	Painter	120000.00
32343	El Said	60000.00	History	Painter	50000.00
58583	Califieri	62000.00	History	Painter	50000.00
15151	Mozart	40000.00	Music	Packard	80000.00
22222	Einstein	95000.00	Physics	Watson	70000.00
33456	Gold	87000.00	Physics	Watson	70000.00

When Data is Jumbled…

- Key issues
  - Repetition of data → increases the size of database
  - *Data consistency issues*
    - Insertion anomaly: Inserting redundant data for every new record
    - Deletion anomaly: Loss of related data, when some data is deleted
    - Update anomaly: When updating certain information, every single record must be updated

ID	name	salary	dept_name	building	budget
76766	Crick	72000.00	Biology	Watson	90000.00
10101	Srinivasan	65000.00	Comp. Sci.	Taylor	100000.00
45565	Katz	75000.00	Comp. Sci.	Taylor	100000.00
83821	Brandt	92000.00	Comp. Sci.	Taylor	100000.00
98345	Kim	80000.00	Elec. Eng.	Taylor	85000.00
12121	Wu	90000.00	Finance	Painter	120000.00
76543	Singh	80000.00	Finance	Painter	120000.00
32343	El Said	60000.00	History	Painter	50000.00
58583	Califieri	62000.00	History	Painter	50000.00
15151	Mozart	40000.00	Music	Packard	80000.00

Solution: Decomposition!

- How to avoid the repetition-of-information problem?

ID	name	salary	dept_name	building	budget
76766	Crick	72000.00	Biology	Watson	90000.00
10101	Srinivasan	65000.00	Comp. Sci.	Taylor	100000.00
45565	Katz	75000.00	Comp. Sci.	Taylor	100000.00
83821	Brandt	92000.00	Comp. Sci.	Taylor	100000.00
98345	Kim	80000.00	Elec. Eng.	Taylor	85000.00
12121	Wu	90000.00	Finance	Painter	120000.00
76543	Singh	80000.00	Finance	Painter	120000.00
32343	El Said	60000.00	History	Painter	50000.00
58583	Califieri	62000.00	History	Painter	50000.00
15151	Mozart	40000.00	Music	Packard	80000.00
22222	Einstein	95000.00	Physics	Watson	70000.00
33456	Gold	87000.00	Physics	Watson	70000.00

- A: Decompose it into two schemas (as they were)
  - Normalization = decomposition of relational schemas
  - Key idea: split relational schemas such that only directly related data composes a relation

Decomposition

- Less redundancy → Uses smaller disk storage; Causes less issues associated with insertion, deletion, and update anomalies

- Not all decompositions are good
  - *E.g.*, Suppose we decompose

    employee(ID, name, street, city, salary)

    into

    employee1 (*ID*, *name*)
    employee2 (*name*, *street, city, salary*)

    → Problem: *What if there are two employees with the same name?*

- Lossy decomposition: a decomposition from which the original relation cannot be reconstructed

Lossy Decomposition

ID	name	street	city	salary
⋮				
57766	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000
⋮				

employee

ID	name
⋮	
57766	Kim
98776	Kim
⋮	

name	street	city	salary
⋮			
Kim	Main	Perryridge	75000
Kim	North	Hampton	67000
⋮			

natural join

ID	name	street	city	salary
⋮				
57766	Kim	Main	Perryridge	75000
57766	Kim	North	Hampton	67000
98776	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000
⋮				

Lossless Decomposition

- Let *R* be a relation schema and let R_1 and R_2 form a decomposition of R; that is, R = $R_1 \cup R_2$
- A decomposition is a lossless decomposition if there is no loss of information by replacing R with the two relation schemas $R_1 \cup R_2$
  - Formally, $\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B

  - *C.f.*, Conversely, a decomposition is lossy if when the join of the projection results is computed, a proper superset of the original relation is returned

$$r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

Normalization

- Database normalization: Process of structuring a database to reduce data redundancy and improve data integrity
  - In accordance with a series of normal forms (next topic)
  - Through the process:
    - One can decompose relations to suppress data anomalies
    - One can make sure the decomposition is lossless

Agenda

- Motivating example
- **Normal forms**

Normal Forms

- Normalization process
  - Normalization is a database design technique, which is used to design a relational database table up to higher normal form
    - Procedurally separates logically independent (but related) data entities into multiple relations
    - Maintains the connections using keys

  - Progressive process
    - A higher level of database normalization cannot be achieved unless the previous levels have been satisfied
      - UNF: Unnormalized form
      - 1NF: First normal form
      - 2NF: Second normal form
      - 3NF: Third normal form
      - BCNF: Boyce-Codd normal form (3.5NF)
      - 4NF: Fourth normal form
      - …

Normal Forms

- Normal forms are backed by a set of normalization theories
  - *Functional dependencies*
    - *Partial dependencies*
    - *Transitive dependencies*
  - *Multi-valued dependencies*

- These theories decide whether a particular relation *R* is in "good form"
  - For a relation *R* is not in "good form", decompose it into a set of relations $\{R_1, R_2, \ldots, R_n\}$ such that
    - Each relation is in good form
    - The decomposition is a lossless decomposition

First Normal Form (1NF)

- Requirements
  - A relation should consist of atomic values
    - Atomic value: a value that cannot be divided (≃primitive data types in JAVA)
      - Atomic - INT, FLOAT, DOUBLE, DECIMAL (NUMERIC), CHAR, VARCHAR, BLOB, TEXT
      - NOGO - Structure, List (array)
  - Attributes should have unique identifiers

- Step 1 of the normalization process
  - *"If the tables in your DB does not follow 1NF, stop using database"*

First Normal Form (1NF)

- 1NF checklist
  1. Each column should contain an atomic value
     - Entries like (x, y) violate this rule
  2. Each column should contain values that are in the same data domain
     - Do not mix different types of values in a column
  3. Each column should have a unique name
     - Duplicate names lead to confusion while accessing data
  4. The order in which data is stored does not matter
     - Using SQL, one can easily fetch data in any order
  5. There are no duplicated rows in the table
     - Primary key (PK) ensures:
       - Attributes that are part of PK are unique
       - Attributes that are part of PK are not null

Functional Dependencies

- There are usually a variety of constraints (rules) on the data in the real world
  - *E.g.*, some of the constraints that are expected to hold in the *university* database are:
    - *Students* and *instructors* are *uniquely identified* by their ID
    - Each *student* and *instructor* has *only one* name
    - Each *instructor* and *student* is associated with *only one* department
    - Each *department* has *only one* value for its budget, and *only one* associated *building*
  - Relations require that the value for a certain set of attributes determines uniquely the value for another set of attributes
    - A functional dependency is a generalization of the notion of a key (= trivial dependency)

Functional Dependencies

- Let R be a relation schema, α and β be its attributes ($\alpha \subseteq R$ *and* $\beta \subseteq R$)
- The functional dependency $\alpha \rightarrow \beta$ holds on R if and only if, for any relations r(R), whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β
  - That is, $t_1[\alpha] = t_2[\alpha] \quad \Rightarrow \quad t_1[\beta] = t_2[\beta]$
  - *E.g.*, Consider r(A,B) with the following instance of r

A	B
1	4
1	5
3	7

    - On this instance, $B \rightarrow A$ hold; $A \rightarrow B$ does **NOT** hold (only the values of B are unique)

Keys and Functional Dependencies

- Functional dependencies allow us to express constraints that cannot be expressed using super keys

  - *E.g.*, consider the schema:

    in_dep (*ID*, *name, salary,* *dept_name*, *building, budget*)

    - *ID* = *instructor* ID
    - *dept_name* = department of the instructor

  - We expect the following functional dependencies to hold:
    - ***dept_name*** $\rightarrow$ ***building***
    - ***ID*** $\rightarrow$ ***building***

    $\Rightarrow$ *dept_name and ID are* *super keys* = *candidate keys*

    * *ID*, *dept_name*, {*ID, dept_name*}

  - We would not expect the next to hold:
    - *dept_name* $\rightarrow$ *salary*

Functional Dependencies

- For relation $R = a_1\ a_2\ a_3\ a_4$

Candidate key = a1, a2, a3, {a1, a2}, {a2,a3}, {a1, a3}, {a1,a2,a3}

  - a_1 is a part of the key
    - Functional dependency: a_4 depends on its key, $a_1a_2a_3$
      - b_id, b_name, bd_detail_attrib1, , bd_detail_attrib2
        - Candidate keys: b_id, b_name, {b_id, b_name}

    - It should not be such that a_4 depends on a_1, and does not depend on $a_1a_2a_3$ ($a_1 \rightarrow a_4$) $\Rightarrow$ partial dependency

Trivial Functional Dependencies

- *A* functional dependency is trivial if it is satisfied by all instances of a relation
  - Example:
    - *ID, name* $\rightarrow$ *ID*
    - *name* $\rightarrow$ *name*
  - In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

First Normal Form (1NF)

- Example

student_id	***name***	***course***
21800999	James Inexistente	Algorithm, OS
21800998	Mike Inexistente	Java
21800997	Matt Inexistente	Algorithm, DB

First Normal Form (1NF)

- Example

student_id	***name***
21800999	James Inexistente
21800998	Mike Inexistente
21800997	Matt Inexistente

student_id	***course***
21800999	Algorithm
21800999	OS
21800998	Java
21800997	Algorithm
21800997	DB

First Normal Form (1NF)

- Example

student_id	name
21800999	James Inexistente
21800998	Mike Inexistente
21800997	Matt Inexistente

student_id	course
21800999	Algorithm
21800999	OS
21800998	Java
21800997	Algorithm
21800997	DB

Second Normal Form (2NF)

- Requirements
  - A relation should be in 1NF (normal forms should be applied in order)
  - A relation should NOT have a non-PK that is functionally dependent on any subset of any candidate key = NO PARTIAL DEPENDENCIES!
    - Any attributes other than PK should be dependent on PK
    - It should not have partial dependencies
    - PK: Primary Key
      - An attribute or a set of attributes that uniquely identifies each tuple in a relation
      - A PK can fetch data from any specific data in a relation
      - *E.g.*, get the department name of *student_ID* = 21800999

Partial Dependencies

- For relation $R = \underbrace{a_1\ a_2\ a_3}_{\textit{Primary key (Composite)}}\ a_4$

  - a_1 is a part of the primary key
    - Partial dependency: a_4 depends on a_1, and does not depend on $a_1 a_2 a_3$
      - Key: $a_1\ a_2\ a_3$
      - Dependency: $a_1 \rightarrow a_4$

Eliminating Partial Dependencies

- Example: *teaches2*(*ID*, *course_id*, *sec_id*, *semester*, *year*, *name*)
  - *ID* → *name*

ID	course_id	sec_id	semester	year	name
10101	CS-101	1	Fall	2,017	Srinivasan
10101	CS-315	1	Spring	2,018	Srinivasan
10101	CS-347	1	Fall	2,017	Srinivasan
12121	FIN-201	1	Spring	2,018	Wu
15151	MU-199	1	Spring	2,018	Mozart
22222	PHY-101	1	Fall	2,017	Einstein
32343	HIS-351	1	Spring	2,018	El Said
45565	CS-101	1	Spring	2,018	Katz
45565	CS-319	1	Spring	2,018	Katz
76766	BIO-101	1	Summer	2,017	Crick
76766	BIO-301	1	Summer	2,018	Crick
83821	CS-190	1	Spring	2,017	Brandt
83821	CS-190	2	Spring	2,017	Brandt
83821	CS-319	2	Spring	2,018	Brandt
98345	EE-181	1	Spring	2,017	Kim

Eliminating Partial Dependencies

- Example:
  - *teaches2a*(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>)
  - *teaches2b*(<u>*ID*</u>, *name*)

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2,017
10101	CS-315	1	Spring	2,018
10101	CS-347	1	Fall	2,017
12121	FIN-201	1	Spring	2,018
15151	MU-199	1	Spring	2,018
22222	PHY-101	1	Fall	2,017
32343	HIS-351	1	Spring	2,018
45565	CS-101	1	Spring	2,018
45565	CS-319	1	Spring	2,018
76766	BIO-101	1	Summer	2,017
76766	BIO-301	1	Summer	2,018
83821	CS-190	1	Spring	2,017
83821	CS-190	2	Spring	2,017
83821	CS-319	2	Spring	2,018
98345	EE-181	1	Spring	2,017

ID	name
10101	Srinivasan
10101	Srinivasan
10101	Srinivasan
12121	Wu
15151	Mozart
22222	Einstein
32343	El Said
45565	Katz
45565	Katz
76766	Crick
76766	Crick
83821	Brandt
83821	Brandt
83821	Brandt
98345	Kim

Eliminating Partial Dependencies

- Example: *score*(*score_id*, <u>*student_id*</u>, <u>*subject_id*</u>, *score*, *instructor*)
  - *subject_id* → *instructor : partial dependency*

score_id	***student_id***	***subject_id***	***score***	***instructor***
1	10	1	82	James Packer
3	11	1	95	James Packer
2	10	2	77	Cole Miller
4	11	2	71	Cole Miller
5	11	4	96	Adam Lee

Eliminating Partial Dependencies

- Example: *score_a*(*score_id*, <u>*student_id*</u>, <u>*subject_id*</u>, *score*)
  score_b(<u>*subject_id*</u>, *instructor*)

score_id	***student_id***	***subject_id***	***score***
1	10	1	82
3	11	1	95
2	10	2	77
4	11	2	71
5	11	4	96

subject_id	***instructor***
1	James Packer
2	Cole Miller
4	Adam Lee

Third Normal Form (3NF)

- Requirements
  - A relation should be in 2NF
  - A relation should NOT have transitive dependencies
    - Transitive dependency: A non-PK attribute depends on another non-PK attribute or a set of non-PK attributes

Transitive Dependencies

- For relation $R = a_1\ a_2\ a_3\ a_4$

Candidate key = a1, a2, {a1,a2}

  - a_1 is the primary key in R
    - a_3 depends on a_1 ($a_1 \rightarrow a_3$) -- OK
    - a_4 depends on a_3 ($a_1 \rightarrow a_3 \rightarrow a_4$) … non-PK → non-PK

Eliminating Transitive Dependencies

- Example

BookNo	Patron	Address	Due
B1	J. Fisher	101 Main Street	3/2/15
B2	L. Perez	202 Market Street	2/28/15

Eliminating Transitive Dependencies

- Example

BookNo	Patron	Address	Due
B1	J. Fisher	101 Main Street	3/2/15
B2	L. Perez	202 Market Street	2/28/15

  - PK: BookNo
  - Patron → Address

Eliminating Transitive Dependencies

- Example

BookNo	Patron	Due
B1	J. Fisher	3/2/15
B2	L. Perez	2/28/15

Patron	Address
J. Fisher	101 Main Street
L. Perez	202 Market Street

Eliminating Transitive Dependencies

- Example

Tournament	Year	Winner	DOB
Indiana Invitational	1998	Al Fredrickson	21 July 1975
Cleveland Open	1999	Bob Albertson	28 Sept. 1968
Des Moines Masters	1999	Al Fredrickson	21 July 1975

  - PK: {Tournament, Year}
  - Winner → DOB

Boyce-Codd Normal Form (BCNF) = 3.5NF

- Requirements
  - A relation should be in 3NF
  - For any dependency A → B, A should be a super key
    - For A → B, if A is non-PK, then it is NOT in BCNF

Boyce-Codd Normal Form (BCNF)

- Example: *takes2*(<u>*student_id*</u>, <u>*subject*</u>, *instructor*)

<u>*student_id*</u>	<u>*subject*</u>	***instructor***
21800999	C++	Dr. Cpp
21800999	Java	Dr. Java
21800998	C++	Dr. C
21800997	Python	Dr. Python
21800996	C++	Dr. Cpp

- (student, subject) → instructor
- Instructor → subject
  - A non-PK identifies a member of PK: Not in BCNF

Boyce-Codd Normal Form (BCNF)

- Example: *takes2a*(<u>*student_id*</u>, *section_id*),
  takes2b(<u>*section_id*</u>, *subject*, *instructor*)

student_id	***section_id***
21800999	101
21800999	103
21800998	102
21800997	104
21800996	101

section_id	***subject***	***instructor***
101	C++	Dr. Cpp
103	Java	Dr. Java
102	C++	Dr. C
104	Python	Dr. Python

Boyce-Codd Normal Form (BCNF)

- Example: *takes2a*(<u>*student_id*</u>, <u>*section_id*</u>),
  takes2b(<u>*section_id*</u>, *subject*, *instructor*)

student_id	***section_id***
21800999	101
21800999	103
21800998	102
21800997	104
21800996	101

section_id	***subject***	***instructor***
101	C++	Dr. Cpp
103	Java	Dr. Java
102	C++	Dr. C
104	Python	Dr. Python

Fourth Normal Form (4NF)

- Requirements
  - A relation should be in BCNF
  - A relation should NOT have multi-valued dependency
    - Multi-valued dependency occurs due to a bad DB schema

    - Multi-valued dependency occurs when a relation has more than 3 attributes
    - For a relation with attributes A, B, C
      - having dependency, A → B, and
      - B and C are independent from each other

        ➔ Then, the relation may have a multi-valued dependency

Multi-valued Dependencies

- Example
  - *student_id* → *course*
  - *student_id* → *activity*
  - *course* ⊥ *activity* (independent)

student_id	***course***	***activity***
21800999	Statistics	Soccer
21800999	Linear algebra	Basketball
21800999	Statistics	Basketball
21800999	Linear algebra	Soccer

Multi-valued Dependencies

- Example

student_id	***course***
21800999	Statistics
21800999	Linear algebra
21800998	Physics
21800998	Programming 101

student_id	***activity***
21800999	Soccer
21800999	Basketball
21800998	Pool
21800997	Soccer

Summary: Normal Forms

- Theories – Normal forms

Theory	Key Idea	Normal Form
Functional dependency	(PK → non-PK)	2NF
Partial dependency	Part of PK → non-PK	2NF
Transitive dependency	Non-PK → non-PK	3NF
-	Non-PK → PK	BCNF
Multi-valued dependency		4NF
Join dependency		5NF

Overall DB Design Process

- Let us assume schema *R* is given:

  (E-R Model)

  - *R* could have been generated when converting E-R diagram to a set of tables

  (Normalization)

  - *R* could have been a single relation containing *all* attributes that are of interest (called universal relation)
  - Normalization breaks *R* into smaller relations

  (Mixed)

  - *R* could have been the result of some ad-hoc design of relations, which we then test/convert to normal form

E-R Model and Normalization

- When an E-R diagram is carefully designed, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization

- However, in a real (imperfect) design, there can be functional dependencies from non-key attributes of an entity to other attributes of the entity
  - *E.g.*, an *employee* entity with
    - attributes
      department_name and *building*
    - functional dependency
      department_name $\rightarrow$ *building*
    - Good design would have made *department* an entity

- Functional dependencies from non-key attributes of a relationship set possible, but rare --- most relationships are binary

Denormalization for Performance

- We may want to use non-normalized schema for performance
  - For example, displaying *prereqs* along with *course_id,* and *title* requires join of *course* with *prereq*

- Alternative 1: Use denormalized relation containing attributes of *course* as well as *prereq* with all above attributes
  - faster lookup
  - extra space and extra execution time for updates
  - extra coding work for programmer and possibility of error in extra code

- Alternative 2: Use a materialized view defined a *course* ⋈ *prereq*
  - Benefits and drawbacks same as above, except no extra coding work for programmer and avoids possible errors

Remaining Issues

- Some aspects of database design are not caught by normalization

- Example (to be avoided)
  - *earnings_2004, earnings_2005, earnings_2006*, etc., all on the schema (*company_id, earnings*)
    - Above are well normalized (in BCNF), but make querying across years difficult and needs new table each year
  - *company_year* (*company_id, earnings_2004, earnings_2005, earnings_2006*)
    - Above are well normalized (in BCNF), but makes querying across years difficult and requires new attribute each year

  - This is an example of a crosstab, where values for one attribute become column names

  → Better schema: *earnings* (*company_id, year, amount*)